

Low Cooling Access: Method Note

Low cooling access

Plain-language purpose

This layer shows where the modeled network has the weakest route to a cooling-supporting cell. It is a network-access proxy, not a census of air-conditioning, tree ownership, or lived experience.

Inputs and network

The analysis uses the same raster graph as the Cheeger layer. Each edge has conductance w_{ij} ; its travel cost is $\text{cost}_{ij} = \text{length}_{ij} / w_{ij}$. High-conductance connections therefore cost less to traverse in the model.

Cooling sinks

The pipeline identifies candidate cooling sinks from the normalized vegetation and temperature fields. A zero-cost super-sink is connected to every identified cooling sink. This converts “distance to any cooling sink” into one shortest-path calculation.

Access score

For each cell i , Dijkstra’s algorithm finds the least accumulated network cost $d(i, \text{sink})$. The map normalizes the negative distance:

```
cooling_access(i) = normalize(-d(i, sink)) .
```

Thus higher values mean easier modeled access; the map’s **Low cooling access** layer highlights the lower end of that score. The exported values are displayed on a 0–100 scale.

What the score includes – and omits

It includes modeled thermal gradients, neighborhood connectivity, and optional greenness through the graph weights. It does not directly include building access, opening hours, cost, disability access, safety, social trust, private cooling, or public-program eligibility.

Responsible use

Use it to ask “where should we learn more about relief?” Validate against locally maintained cooling-center, tree-canopy, transit, and community knowledge. Do not use it as a claim that residents lack cooling, or as the sole basis for allocating resources.

Reproducibility

Implementation: `core/graph.py` and `core/pipeline.py` (`_sink_nodes` , `_distance_to_sinks`). Report the sink rule, inputs, resolution, graph connectivity, `alpha` , and `beta` alongside the output.